人工智能学院学院 大数据管理与分析 课程实验报告

实验题目：实验二　文档倒排索引算法实现		学号：202400390057
日期：2026.10.14	班级： 2 班	姓名： 李丰润
Email：202400390057@mail.sdu.edu.cn		

实验目的：

1. 理解倒排索引的概念与结构：以“单词→ 出现该单词的文档列表”为核心映射，掌握词项词典与倒排记录表两部分及其按文档编号有序存储的意义；

2. 掌握用 MapReduce 构建倒排索引的方法：Map 阶段分词、去停词并输出“单词-文档”对，Combine 阶段在 Map 端局部聚合，Reduce 阶段按单词汇总各文档词频，体会“分而治之 + 按 key 聚合”的编程思想；

3. 熟练掌握 MapReduce 程序在 Hadoop 集群（伪分布式）上的完整流程：编写、编译、打包、上传数据到 HDFS、提交作业、查看运行统计与结果；

4. 学会从公开网页采集英文文档语料、构建并分发停词表，并对倒排索引结果进行单词检索验证。

实验软件和硬件环境：

硬件环境：x86_64 个人计算机；Oracle VM VirtualBox 7.1.8 虚拟机 lfr6（Ubuntu 64 位，内存 2 GB）

软件环境：Ubuntu 22.04 LTS（64 位）；OpenJDK 1.8.0_504；Apache Hadoop 3.3.6（伪分布式，HDFS + YARN，NameNode 与 ResourceManager 运行于本机）；辅助工具 wget（下载文档）、Python 3.10（网页 HTML 转纯文本）。

实验代码与数据位于：https://github.com/mhy668/bigData

实验原理和方法：

1. 倒排索引的原理

正排索引以文档为主键，记录“一篇文档包含哪些词”；而全文检索需要的是反向映射：给定一个查询词，快速找到包含它的全部文档。倒排索引正是为此设计的索引结构：以词项（term）为主键，为每个词项维护一张“倒排记录表”（posting list），记录该词出现过的文档及词频，通常按文档编号升序存储，便于压缩和后续多词项归并。本实验要求输出的格式为：data <document01.txt,6>;<document02.txt,3>;<document05.txt,2>;<total,11>.，查询某个单词时只需在结果文件中定位该词对应的一行，即可得到它出现在哪些文档、各出现多少次。

2. 基于 MapReduce 的构建方法

大规模语料的倒排索引构建本质上是“按词项分组求和”的聚合问题，天然

适合 MapReduce 模型，程序由 Mapper、Combiner、Reducer 三部分组成。

（1）Mapper（分词与过滤）：以“行号—行内容”为输入，通过 FileSplit 取到当前分片的文件名（即文档名）；行内容统一小写，非字母数字字符替换为空格后分词；过滤长度小于 2、以数字开头、位于停词表中的词；对保留的每个词项输出 <单词, “文档名:1”>，即一条“该词在该文档出现 1 次”的部分倒排记录。

（2）Combiner（Map 端局部聚合）：在 Map 端先把同一文档内同一单词的计数合并为“文档名:次数”，例如 (hadoop, document01.txt:1) 出现多次合并为 (hadoop, document01.txt:91)。Combiner 本质是“局部的 Reduce”，可把 Shuffle 传输量压缩数倍（本实验 73623 条 Map 输出记录聚合为 12384 条）。

（3）Reducer（全局汇总）：框架按单词分组后，Reducer 把该单词的全部“文档名:次数”对累加到 TreeMap（按文档名自动升序）中并累计总词频 total，拼接成 word <文档 1,次数>;...;<total,总数>. 的格式输出。停词表（179 个 a、an、the、and、is 等高频词）上传到 HDFS，由 Mapper 在 setup() 阶段读入内存（每个 Map 任务只读一次），分词时逐词过滤。

3. 关键伪代码

```
Mapper(行号 key, 行内容 value):
    docName <- 当前输入分片的文件名        // FileSplit.getPath().getName()
    tokens  <- 分词(小写化(value), 非字母数字字符 -> 空格)
    for token in tokens:
        if token 是停词 or 长度 < 2 or 首字符是数字: continue
        emit(token, docName + ":1")       // 部分倒排记录：该词在该文档出现 1 次

Combiner(word, ["doc:1", "doc:1", ...]):  // 局部的、可结合的聚合，压缩 Shuffle 数据量
    按文档名累加计数
    emit(word, "doc:cnt")                 // 例："document01.txt:91"

Reducer(word, ["doc1:16", "doc2:75", ...]):
    perDoc <- TreeMap<文档名, 词频>        // 红黑树，自动按文档名升序排列
    total  <- 0
    for "doc:cnt" in values:              // 累加各文档词频
        perDoc[doc] += cnt; total += cnt
    emit(word, "<doc1,c1>;<doc2,c2>;...;<total,total>.")
```

实验步骤：（不要求罗列完整源代码）

全部操作在虚拟机 lfr6（Ubuntu 22.04）的图形界面终端中完成，实验目录为 ~/bigdata-lab2，各步骤整理为 scripts/ 目录下的脚本，下列截图为实际运行画面。

（1）下载英文文档（12 篇）

用 wget 从公开网页下载 12 篇英文文档（Apache Hadoop/Spark 技术文档、Python HOWTO、Git 电子书、MDN 文档、GNU 手册与 RFC 791/1035/2616 标准文本），HTML 网页去标签后保存为 document01.txt ~ document12.txt，文档描述如下表：

编号	文件名	来源（公开网页）	词数
1	document01.txt	Apache Hadoop - MapReduce Tutorial	8239
2	document02.txt	Apache Hadoop - HDFS Architecture	4711
3	document03.txt	Apache Hadoop - YARN	848
4	document04.txt	Apache Spark - Quick Start	2313
5	document05.txt	Python - Sorting HOWTO	2127
6	document06.txt	Python - Regular Expression HOWTO	8607
7	document07.txt	Git - About Version Control	1322
8	document08.txt	MDN - What is JavaScript	5022
9	document09.txt	GNU Coreutils - Introduction	311
10	document10.txt	RFC 791 Internet Protocol	11192
11	document11.txt	RFC 1035 Domain Names	15901
12	document12.txt	RFC 2616 HTTP/1.1	57897
合计	12 篇	——	118490

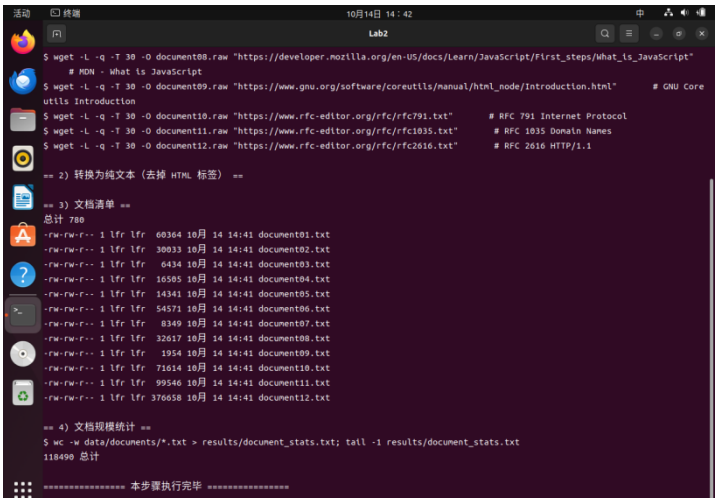


图 1　步骤 1：下载 12 篇英文文档、转换为纯文本并统计词数

（2）建立停词表

停词表 data/stopwords.txt 共 179 个英文停用词，一行一词（# 开头为注释），覆盖冠词、介词、代词、常见助动词及缩写形式（the、and、of、is、

don’ t 等），运行时上传到 HDFS 供所有 Map 任务共享读取。

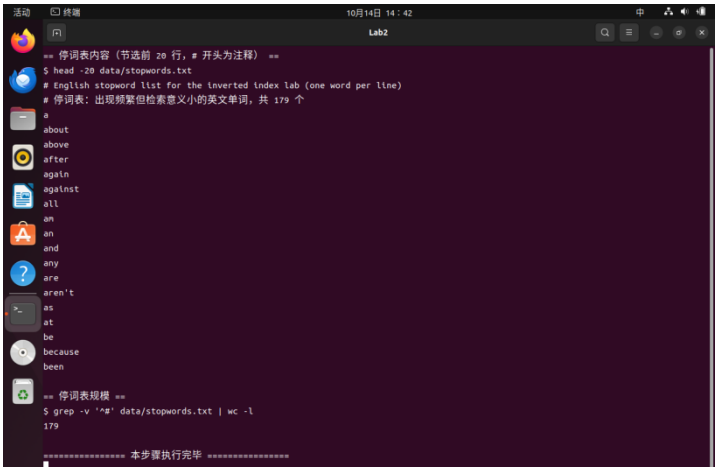


图 2　步骤 2：停词表内容（节选）与词数统计

（3）编译与打包 MapReduce 程序

源码 code/InvertedIndex.java（Mapper/Combiner/Reducer 与 Job 装配），用 Hadoop 的 classpath 编译后打包成 jar：

```
$ javac -encoding UTF-8 -classpath $(hadoop classpath) \
        -d build code/InvertedIndex.java
$ jar cf invertedindex.jar -C build .
```

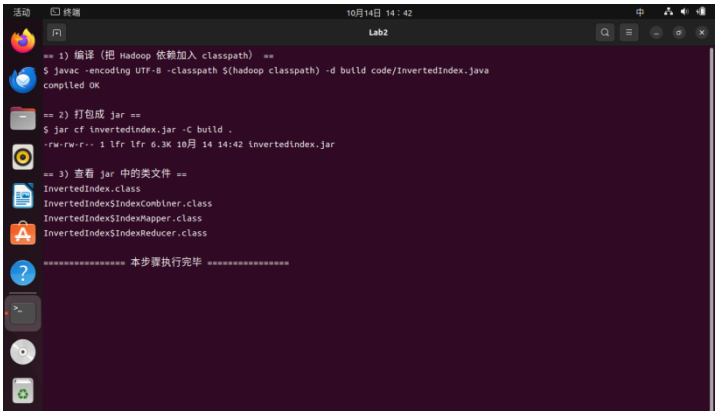


图 3　步骤 3：程序编译成功并打包成 invertedindex.jar

（4）启动 Hadoop 伪分布式环境

启动 HDFS（NameNode、DataNode、SecondaryNameNode）与 YARN（ResourceManager、NodeManager），用 jps 检查进程：

```
$ start-dfs.sh          $ start-yarn.sh          $ jps
```

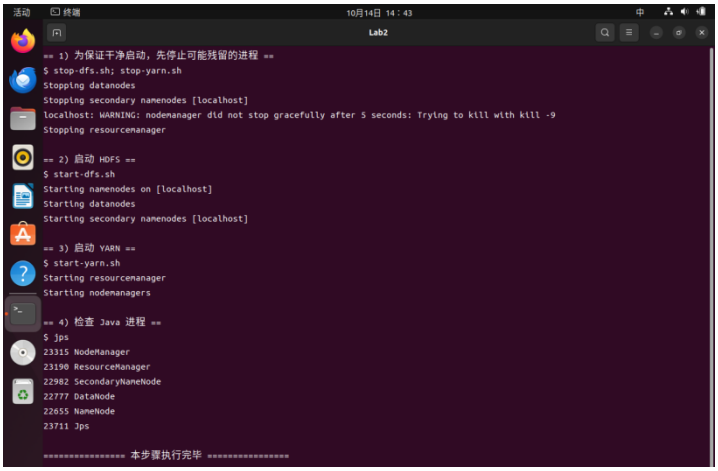


图 4　步骤 4：HDFS 与 YARN 启动成功（jps 显示 5 个守护进程）

（5）上传数据并运行倒排索引作业

把 12 篇文档与停词表上传到 HDFS，然后提交 MapReduce 作业（第 3 个参数为 HDFS 上的停词表路径）：

```
$ hdfs dfs -mkdir -p /user/lfr/lab2/input
$ hdfs dfs -put -f data/documents/*.txt /user/lfr/lab2/input/
$ hdfs dfs -put -f data/stopwords.txt   /user/lfr/lab2/stopwords.txt
$ hadoop jar invertedindex.jar InvertedIndex \
      /user/lfr/lab2/input /user/lfr/lab2/output /user/lfr/lab2/stopwords.txt
```

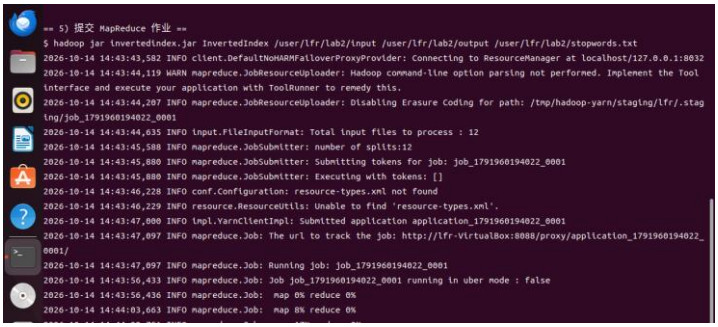


图 5　步骤 5：作业提交成功，12 个 Map 任务并行执行中

12 个 Map 与 1 个 Reduce 任务全部成功，失败的 Shuffle、错误记录数均为 0；Combine 把 73623 条 Map 输出记录局部聚合为 12384 条，最终输出 5869 条倒排索引记录：

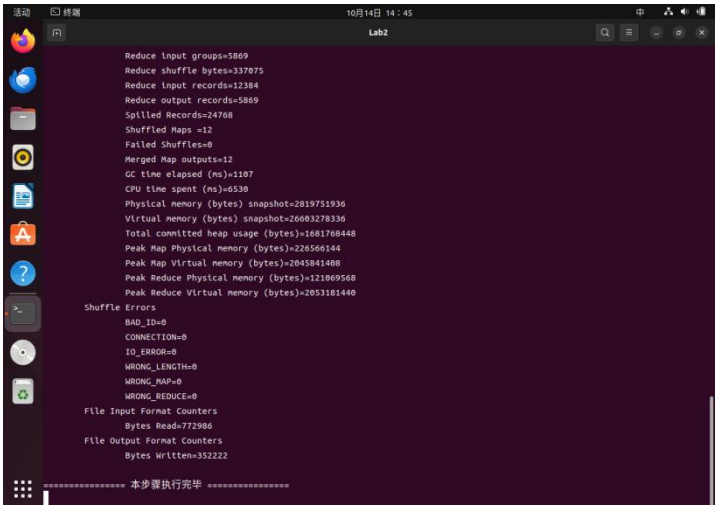


图 6　步骤 5：作业运行统计（Counters）

（6）查看倒排索引结果

作业输出保存在 HDFS 输出目录的 part-r-00000 文件中（352 KB），下载到本地 results/：

```
$ hdfs dfs -ls /user/lfr/lab2/output
$ hdfs dfs -get -f /user/lfr/lab2/output/part-r-00000 results/
$ wc -l results/part-r-00000      # 5869 条索引记录（去重后的词项数）
```

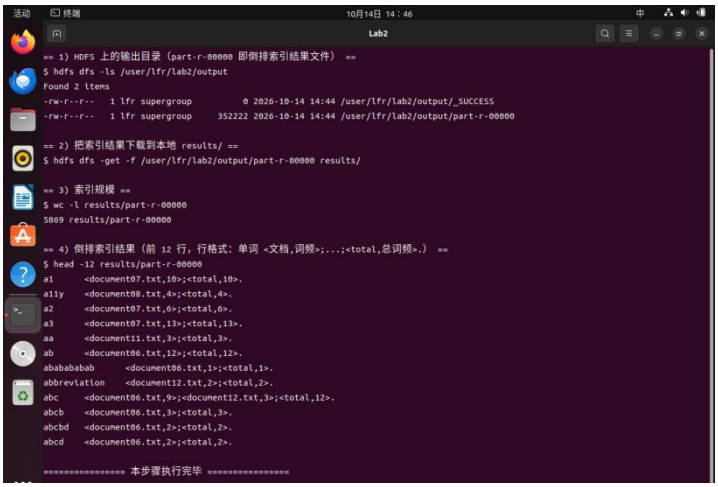


图 7　步骤 6：倒排索引结果（前 12 行，每行一个词项的词典与倒排记录表）

（7）单词检索示例

在结果文件中查询单词，即得到该词出现在哪些文档、各出现多少次。例如查询 data、hadoop、mapreduce、search、index 五个单词：

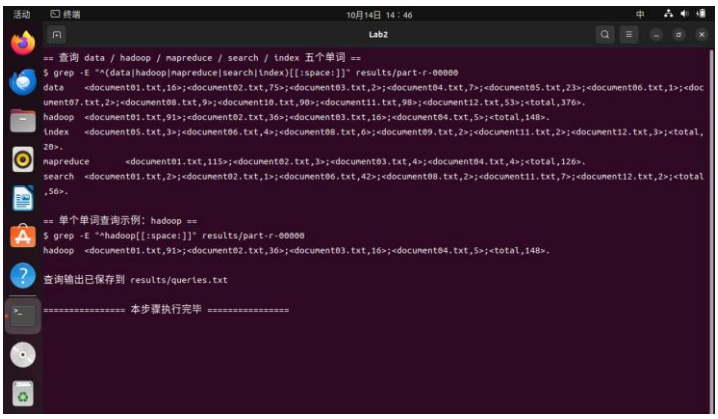


图 8　步骤 7：单词检索示例（grep 查询倒排索引文件）

以 hadoop 为例：hadoop <document01.txt,91>;<document02.txt,36>;<document03.txt,16>;<document04.txt,5>;<total,148>.，说明该词在 4 篇 Hadoop/Spark 文档中共出现 148 次（MapReduce 教程 91 次、HDFS 文档 36 次），与文档主题一致；data 出现于全部 12 篇文档、总计 376 次，mapreduce 126 次，search 56 次，index 20 次。结果与文档语义相符，验证了倒排索引的正确性。

结论分析与体会：

1. 对倒排索引的理解：倒排索引把“文档→词”的正排关系反转为“词→文档列表”，配合词频就有了加权检索的基础，查询时定位单词即可直接读出文档列表，这正是搜索引擎“毫秒级”响应的关键数据结构。本实验 12 篇文档共 11.8 万词，去停词后仍有 5869 个不同词项，若进一步按倒排记录表有序压缩（如差值编码），规模还能更小，为后续索引压缩实验留下空间。

2. 对 MapReduce 编程模型的理解：倒排索引构建被自然分解为“分词打标记（Map）→ 局部聚合（Combine）→ 按词分组汇总（Reduce）”三段，程序员只需编写 Mapper 与 Reducer 的业务逻辑，排序、分组、容错、并行调度全部由框架完成；Shuffle 自动把相同单词的记录汇聚到同一 Reduce 分组，生动体现了“移动计算而不是移动数据”的设计思想。

3. 工程细节上的收获：（1）文档名无法预先获知，必须通过 ctx.getInputSplit() 取 FileSplit 的文件名，这是“以文件名作为文档编号”的关键；（2）Combiner 与 Reducer 聚合逻辑相似但输出形态不同，不能互换；（3）停词表放在 HDFS、经配置传入路径并在 setup() 中一次性读入内存，更新停词表无需重新编译程序；（4）过滤单字符与纯数字（如 RFC 文档中位图串）后，索引噪声明显减少、结果更整洁。总体而言，本实验完整走通了“语料采集→停词表设计→MapReduce 编程→集群运行→检索验证”的全流程。

4. 实验代码、数据与结果已整理至 GitHub 公开仓库：https://github.com/mhy668/bigData（实验二内容位于 bigdata-lab2/ 目录）。

就实验过程中遇到和出现的问题，你是如何解决和处理的，自拟 1－3 道问答题：

问题 1：Reducer 汇总某单词的倒排记录表时，为什么用 TreeMap 而不是 HashMap？

答：同一单词的各文档词频由多个 Map 任务产生，到达 Reducer 的顺序不确定。若使用 HashMap，输出中 <document01.txt,…>、<document02.txt,…> 的顺序会随运行变化，结果不稳定也不易阅读。TreeMap 基于红黑树实现，put/merge 复杂度为 O(log n)，遍历时天然按文档名升序输出，正好满足“倒排记录表按文档编号有序”的要求，也便于人工核对和后续多词项归并（求交集/并集）。

问题 2：为什么本实验的倒排索引作业要设置 Combiner？它与 Reducer 有何区别和联系？

答：Mapper 对每行每个词都输出一条记录，同一单词在同一文档中会被重复输出多次，若全部直接传给 Reducer，Shuffle 阶段的网络传输与磁盘 IO 开销巨大。Combiner 在 Map 端先做一次“局部的 Reduce”，把同一文档内同一单词的计数合并——本实验 73623 条 Map 输出记录被压缩为 12384 条（约 5.9 倍）。二者聚合逻辑相同、都满足结合律与交换律（本实验为求和）；区别在于 Combiner 输入输出类型须与 Mapper 输出一致、只做部分聚合，而 Reducer 看到的是一个单词的全部分区数据，负责最终汇总与格式化输出。

问题 3：停词表为什么做成独立文件并在 Mapper.setup() 中读取，而不是写死在程序中？

答：写死在程序中有三个缺点：修改停词表必须重新编译打包；内容与程序逻辑耦合、不可复用；无法按任务切换不同语言的停词表。本实验把停词表做成文本文件并上传到 HDFS，通过 conf.set("invertedindex.stopwords.path", …) 传入路径，Mapper 在每个 Map 任务开始时于 setup() 中一次性读入内存集合：更新停词表只需替换 HDFS 上的文件，无需改代码；setup() 每任务只执行一次，避免了在 map() 中按行反复读文件的巨大 IO 开销。